



Université Euromed de Fès

École d'Ingénierie Digitale et Intelligence Artificielle

Rapport du Travail Pratique : *Cloud*

PostgreSQL - S3 - Snowflake - dbt - Airflow - BI

Préparé par : Marwa Hanini
Nada Ezzouhour El Achabi
Reda Jelfa
Mostapha Oulghazi

Table des matières

1	Introduction	2
1.1	Contexte du projet ShopStream	2
1.2	Objectifs du pipeline	2
2	Architecture globale du pipeline	4
2.1	Vue d'ensemble de l'architecture du pipeline	4
2.2	Description détaillée de l'architecture par couches	4
3	Description technique des composants du pipeline	7
3.1	PostgreSQL : source transactionnelle	7
3.1.1	Rôle	7
3.1.2	Schéma de données	7
3.1.3	Génération des données	7
3.2	Data Lake : Amazon S3	7
3.2.1	Rôle	7
3.2.2	Organisation des données	8
3.2.3	Scripts d'export	9
3.3	Snowflake : Data Warehouse	9
3.3.1	Rôle	9
3.3.2	Configuration	9
3.3.3	Chargement des données	10
3.4	dbt : transformations analytiques	10
3.4.1	Rôle	10
3.4.2	Modèles	10
3.4.3	Tests	12
3.4.4	Documentation	12
3.5	Airflow : orchestration du pipeline	14
3.5.1	Rôle	14
3.5.2	Concept de DAG	14
3.5.3	Structure du DAG ShopStream	15
3.5.4	Gestion des dépendances et des échecs	18
3.5.5	Planification et automatisation	18
3.5.6	Apport d'Airflow dans l'architecture globale	19
3.6	Power BI : visualisation et analyse décisionnelle	19
3.6.1	Rôle	19
3.6.2	Connexion aux données	19
3.6.3	Dashboards	19
3.6.4	Mesures DAX	20
3.6.5	Apport de Power BI dans l'architecture globale	20
3.7	Bilan du projet	22
3.8	Difficultés rencontrées	22
3.9	Améliorations possibles	22

1 Introduction

1.1 Contexte du projet ShopStream

ShopStream est une plateforme **SaaS e-commerce fictive** utilisée comme cas d'étude dans le cadre de ce TP. Elle simule le fonctionnement d'une application e-commerce moderne permettant à des marchands de gérer l'ensemble de leur activité en ligne, notamment la gestion des produits, des clients et des commandes, au moyen d'une application transactionnelle centralisée.

L'activité de la plateforme génère en continu un volume important de données opérationnelles. Chaque interaction utilisateur, telle que la création d'un compte, la consultation de produits, la passation d'une commande, le paiement ou encore la navigation sur le site, produit des événements et des enregistrements stockés dans une base de données transactionnelle. Ces données constituent une source précieuse d'information sur le fonctionnement de la plateforme et le comportement des utilisateurs.

Dans un contexte réel, ces données sont d'abord stockées dans une base de données transactionnelle optimisée pour la performance opérationnelle. Toutefois, bien qu'elles soient essentielles au fonctionnement quotidien de l'application, ces données ne sont pas directement exploitables à des fins analytiques avancées. Les entreprises ont en effet besoin de transformer ces données brutes en indicateurs synthétiques afin de suivre leur activité, d'évaluer leurs performances et de soutenir la prise de décision stratégique.

C'est précisément dans ce contexte que s'inscrit le projet ShopStream. Il vise à reproduire une problématique couramment rencontrée en entreprise : le passage de données opérationnelles issues d'un système transactionnel vers une architecture analytique permettant l'analyse du chiffre d'affaires, la mesure de la performance des produits, l'étude du comportement des utilisateurs et l'optimisation des processus métier. La mise en place d'un **pipeline de données moderne** devient alors indispensable pour répondre à ces besoins analytiques.

1.2 Objectifs du pipeline

L'objectif principal du pipeline de données mis en place pour le projet ShopStream est de **transformer des données opérationnelles brutes en informations décisionnelles exploitables**. Ce pipeline permet d'assurer un cheminement structuré et automatisé des données, depuis leur génération par l'application jusqu'à leur visualisation sous forme d'indicateurs clairs et interprétables.

Plus précisément, le pipeline vise à répondre aux objectifs suivants :

- collecter de manière fiable les données transactionnelles issues de la base PostgreSQL, qui représente la source de vérité opérationnelle de l'application ;
- centraliser les données brutes dans un **Data Lake** basé sur Amazon S3, afin d'assurer leur conservation durable, leur traçabilité et leur réutilisation potentielle ;
- charger et structurer ces données dans un **Data Warehouse analytique** (Snowflake), optimisé pour l'exécution de requêtes analytiques complexes ;
- appliquer les règles métier et les transformations nécessaires à l'aide de **dbt**, afin de produire des tables analytiques cohérentes, fiables et prêtes pour l'analyse ;
- automatiser et orchestrer l'ensemble des étapes du pipeline grâce à **Apache Airflow**, garantissant ainsi la reproductibilité et la fiabilité des traitements ;
- exposer les résultats finaux à travers des **tableaux de bord Power BI**, permettant aux utilisateurs métiers de suivre les indicateurs clés de performance et de faciliter la prise de décision.

À travers ces objectifs, le pipeline ShopStream vise à reproduire une architecture de données moderne, proche des standards industriels, mettant en évidence les enjeux de gouvernance des données, de qualité, de performance et d'aide à la décision dans un environnement orienté données.

2 Architecture globale du pipeline

2.1 Vue d'ensemble de l'architecture du pipeline

L'architecture du pipeline de données mise en place pour le projet **ShopStream** repose sur une organisation en **couches fonctionnelles distinctes**, chacune étant responsable d'une étape précise du cycle de vie de la donnée. Cette approche modulaire s'inscrit dans les principes fondamentaux des **architectures de données modernes**, communément regroupées sous le terme de *Modern Data Stack*.

L'objectif principal de cette architecture est de séparer clairement les responsabilités entre les différents composants du système. Chaque couche est spécialisée dans un type de traitement spécifique (transactionnel, stockage brut, analytique, transformation ou visualisation), ce qui permet d'optimiser les performances globales tout en garantissant une meilleure lisibilité de l'architecture.

Cette séparation en couches offre plusieurs avantages majeurs. D'une part, elle assure la **scalabilité** du système : chaque composant peut évoluer indépendamment des autres en fonction des besoins, qu'il s'agisse d'augmenter le volume de données stockées, la puissance de calcul ou le nombre d'utilisateurs accédant aux tableaux de bord. D'autre part, elle améliore la **maintenabilité**, car une modification ou une évolution dans une couche donnée (par exemple le changement d'outil de visualisation) n'entraîne pas de remise en cause de l'ensemble du pipeline.

Par ailleurs, cette organisation contribue à la **robustesse** du pipeline. En cas de dysfonctionnement d'un composant, les autres couches restent opérationnelles et les données peuvent être conservées ou rejouées à partir des étapes intermédiaires. Cette capacité de reprise est essentielle dans un contexte industriel où la fiabilité des données est un enjeu critique.

Le pipeline de données ShopStream suit un flux principal linéaire et contrôlé, allant de la production des données transactionnelles vers leur exploitation analytique. Ce flux est représenté comme suit :

PostgreSQL → Amazon S3 → Snowflake → dbt → Power BI

Ce cheminement illustre le passage progressif de **données opérationnelles brutes**, générées par l'activité de l'application, vers des **données structurées, nettoyées et enrichies**, destinées à l'analyse décisionnelle. Chaque étape du pipeline apporte une valeur ajoutée spécifique : la collecte fiable des données, leur conservation durable, leur structuration analytique, l'application de la logique métier et, enfin, leur restitution sous forme d'indicateurs compréhensibles par les utilisateurs métiers.

Ainsi, cette architecture globale constitue une base solide pour l'analyse des performances de la plateforme ShopStream. Elle reproduit des schémas d'architecture largement utilisés en entreprise et permet de répondre aux exigences actuelles en matière de traitement de données, de gouvernance et d'aide à la décision.

2.2 Description détaillée de l'architecture par couches

Couche Source (OLTP) : PostgreSQL

La première couche du pipeline correspond à la couche source, représentée par la base de données PostgreSQL. Cette base joue le rôle de système OLTP (Online Transaction Processing) et simule la base de données de production de l'application ShopStream. Elle stocke l'ensemble des données métier générées en temps réel par l'activité

de la plateforme, telles que les informations des utilisateurs, les produits, les commandes, les paiements et les événements applicatifs.

PostgreSQL est optimisée pour gérer un grand nombre de transactions simultanées, avec des opérations fréquentes d'insertion et de mise à jour. Son rôle dans le pipeline se limite volontairement à la gestion opérationnelle des données, afin d'éviter toute surcharge liée aux traitements analytiques. Cette séparation garantit la performance et la stabilité de l'application métier.

Couche Data Lake : Amazon S3

La seconde couche est la couche Data Lake, implémentée à l'aide du service Amazon S3. Cette couche a pour objectif principal de stocker les données brutes extraites de PostgreSQL, sans transformation préalable. Les données y sont généralement enregistrées sous forme de fichiers plats (CSV ou JSON), organisés par domaine fonctionnel et par table.

Amazon S3 joue un rôle essentiel de zone tampon entre la source transactionnelle et le système analytique. Il permet de conserver un historique complet des données, d'assurer leur durabilité et de découpler totalement la base PostgreSQL des traitements en aval. En cas de problème dans le data warehouse ou dans les transformations, les données brutes restent disponibles et peuvent être rechargées à tout moment.

Couche Data Warehouse : Snowflake

La troisième couche correspond à la couche Data Warehouse, assurée par Snowflake. Snowflake est un entrepôt de données analytique de type OLAP (Online Analytical Processing), conçu pour exécuter des requêtes complexes sur de grands volumes de données.

Les données stockées dans Amazon S3 sont chargées dans Snowflake, d'abord dans des tables de staging représentant fidèlement les données brutes, puis dans des schémas analytiques plus structurés. Snowflake se distingue par sa capacité à séparer le stockage du calcul, ce qui permet d'adapter dynamiquement les ressources de calcul en fonction des besoins analytiques, sans impacter le stockage des données.

Cette couche constitue le cœur analytique du pipeline, sur lequel reposent l'ensemble des analyses et indicateurs métiers.

Couche Transformation : dbt

La couche de transformation est assurée par dbt (data build tool). dbt permet d'appliquer la logique métier directement au sein du data warehouse Snowflake, en utilisant exclusivement le langage SQL. Il transforme les tables de staging en tables analytiques finales, telles que des tables de faits et des dimensions.

Grâce à dbt, les règles de calcul (chiffre d'affaires, agrégations, indicateurs de performance) sont centralisées, versionnées et documentées. dbt permet également d'intégrer des tests de qualité des données, garantissant la cohérence et la fiabilité des résultats produits. Cette couche joue un rôle clé dans la transformation de données brutes en données prêtes pour l'analyse décisionnelle.

Couche Orchestration : Airflow

La couche d'orchestration est prise en charge par Apache Airflow. Airflow n'effectue aucun traitement direct sur les données, mais agit comme un chef d'orchestre coordonnant l'exécution des différentes étapes du pipeline.

Il définit l'ordre d'exécution des tâches, gère les dépendances entre elles et permet l'automatisation complète du pipeline. Par exemple, l'extraction des données depuis PostgreSQL ne peut être lancée que si la base est disponible,

et les transformations dbt ne s'exécutent qu'après le chargement réussi des données dans Snowflake. Airflow assure ainsi la fiabilité, la reproductibilité et l'industrialisation du pipeline.

Couche BI : Power BI

La dernière couche est la couche de Business Intelligence, représentée par Power BI. Cette couche est destinée aux utilisateurs métiers et aux décideurs. Power BI se connecte directement aux tables analytiques finales stockées dans Snowflake, sans effectuer de transformations lourdes.

À travers des tableaux de bord interactifs et des indicateurs clés de performance (KPI), Power BI permet de visualiser les résultats des analyses, de suivre l'évolution de l'activité et de faciliter la prise de décision stratégique. Cette couche constitue l'aboutissement du pipeline, en transformant les données traitées en valeur métier concrète.

3 Description technique des composants du pipeline

3.1 PostgreSQL : source transactionnelle

3.1.1 Rôle

PostgreSQL constitue la source transactionnelle principale du pipeline de données ShopStream. Il joue le rôle de base de données OLTP (Online Transaction Processing) et simule la base de données de production de l'application e-commerce. Toutes les données générées par l'activité quotidienne de la plateforme (inscriptions, commandes, événements utilisateurs) y sont stockées en temps réel.

Cette base est optimisée pour la gestion de transactions fréquentes, rapides et concurrentes, garantissant la cohérence et l'intégrité des données métier tout en assurant de bonnes performances pour l'application.

3.1.2 Schéma de données

Le schéma PostgreSQL est normalisé, afin d'éviter la redondance des données et de garantir leur cohérence. Il comprend notamment les tables suivantes :

- `users` : contient les informations relatives aux clients (email, pays, type d'abonnement, dates clés).
- `products` : représente le catalogue des produits proposés par les marchands.
- `orders` : stocke les commandes passées par les utilisateurs.
- `order_items` : détaille les lignes de commande associées à chaque commande.
- `events` : enregistre les événements applicatifs (navigation, clics, erreurs).
- `crm_contacts` : contient les données marketing issues des campagnes CRM.

Les relations entre ces tables sont assurées par des clés primaires et étrangères, ce qui garantit l'intégrité référentielle et la cohérence des données tout au long du pipeline.

3.1.3 Génération des données

Afin de simuler une activité réaliste, les données sont générées à l'aide d'un script Python s'appuyant sur les bibliothèques *faker* et *psycpg2*. Ce script permet de créer un volume significatif de données (utilisateurs, produits, commandes, événements) avec des valeurs crédibles (dates, montants, pays), reproduisant le comportement d'une plateforme e-commerce réelle.

Cette génération de données est essentielle pour tester la robustesse du pipeline et valider les traitements analytiques sur des volumes proches de la réalité.

3.2 Data Lake : Amazon S3

3.2.1 Rôle

Amazon S3 joue le rôle de Data Lake, plus précisément de zone RAW du pipeline. Il sert de zone de stockage intermédiaire entre la base transactionnelle PostgreSQL et le data warehouse Snowflake. Les données y sont stockées sans transformation, dans leur format brut.

Cette approche permet de conserver une copie fidèle des données sources, indépendamment des traitements analytiques ultérieurs.

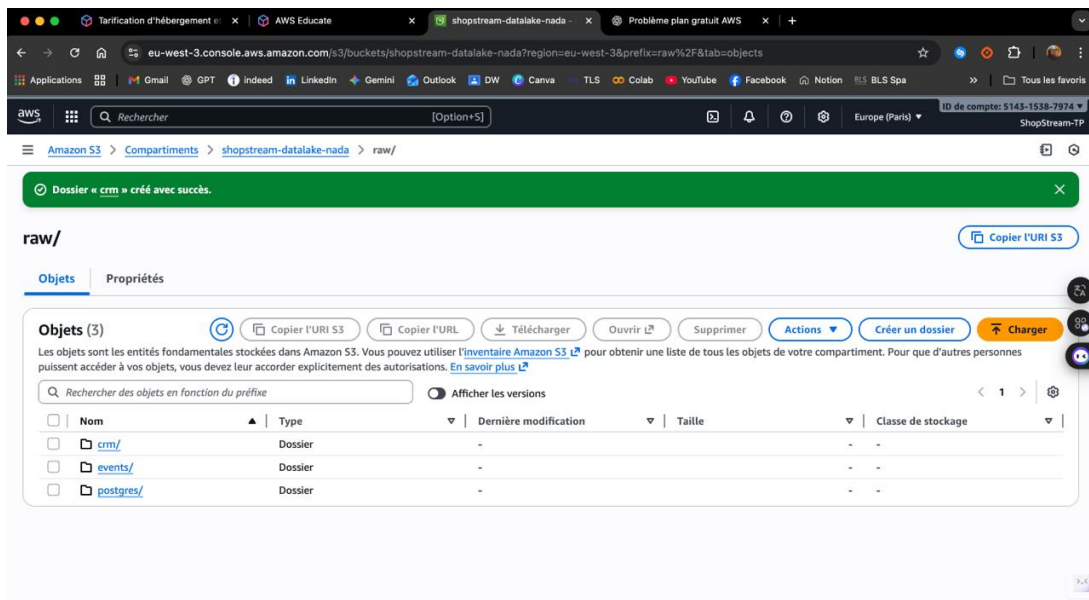
3.2.2 Organisation des données

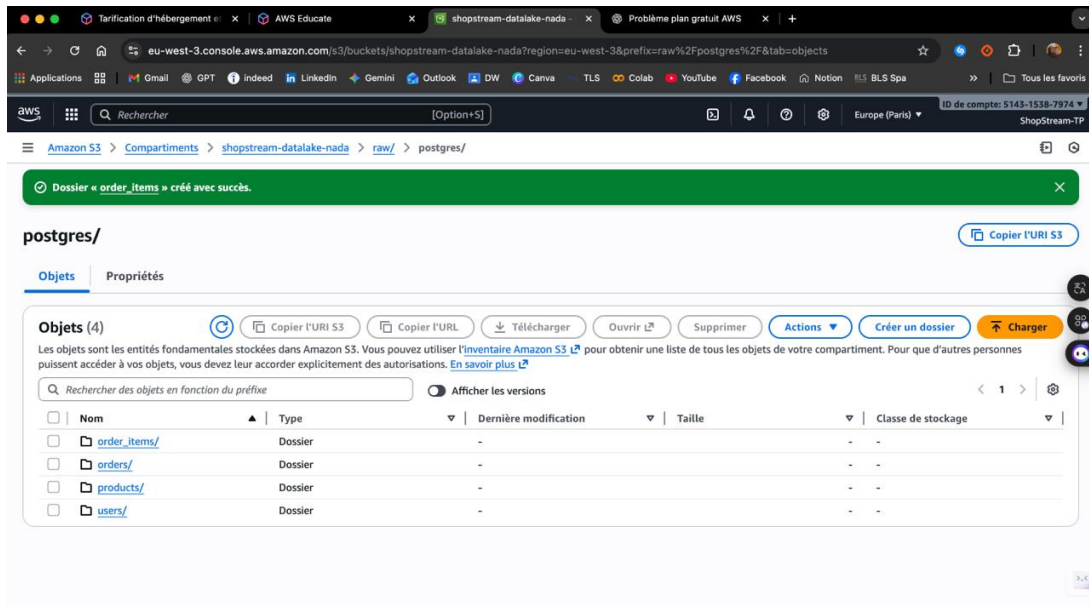
Les données sont organisées de manière hiérarchique, par domaine fonctionnel et par table, afin de faciliter la traçabilité, la maintenance et la reprise des données :

```
raw/  
|- postgres/  
|   |- users/  
|   |- products/  
|   |- orders/  
|   |- order\_items/  
|- events/  
|- crm/
```

Ce choix garantit :

- une lecture claire du contenu du Data Lake,
- une réutilisation des données pour d'autres usages analytiques,
- une récupération rapide en cas d'erreur en aval.





3.2.3 Scripts d'export

L'export des données depuis PostgreSQL vers S3 est réalisé via un script Python. Celui-ci extrait les tables de la base PostgreSQL, les convertit en fichiers CSV, puis les charge dans Amazon S3 à l'aide du SDK *boto3*.

Ce mécanisme d'export assure une séparation claire entre la couche transactionnelle et les couches analytiques, renforçant la robustesse globale du pipeline.

3.3 Snowflake : Data Warehouse

3.3.1 Rôle

Snowflake est utilisé comme **entrepôt de données analytique (Data Warehouse)** au sein du pipeline ShopStream. Il constitue le composant central dédié à l'analyse décisionnelle et à l'exécution de requêtes analytiques complexes sur de grands volumes de données.

Contrairement à PostgreSQL, qui est optimisé pour les usages transactionnels (OLTP), Snowflake est spécifiquement conçu pour les traitements **OLAP (Online Analytical Processing)**, tels que les agrégations massives, les jointures complexes et l'analyse historique des données. Cette spécialisation permet d'obtenir de meilleures performances analytiques sans impacter la base transactionnelle.

3.3.2 Configuration

La configuration de Snowflake dans le cadre de ce projet comprend plusieurs étapes essentielles :

- Création d'une base de données dédiée au projet ShopStream.
- Création de deux schémas distincts :
 - **STAGING** : ce schéma contient les données brutes chargées depuis Amazon S3, sans transformation préalable.
 - **CORE** : ce schéma contient les données transformées, nettoyées et structurées, prêtes à être exploitées pour l'analyse et la visualisation.
- Définition des **warehouses** (entrepôts de calcul), qui permettent d'allouer dynamiquement les ressources de calcul en fonction des besoins analytiques.

Cette séparation entre les zones STAGING et CORE améliore la lisibilité du modèle de données, facilite la maintenance et optimise les performances des requêtes analytiques.

3.3.3 Chargement des données

Le chargement des données dans Snowflake est réalisé à partir d'Amazon S3 à l'aide de commandes `COPY INTO`. Ces commandes permettent d'importer efficacement les fichiers stockés dans le Data Lake vers les tables Snowflake.

Les tables du schéma **STAGING** reflètent fidèlement les données brutes issues des fichiers CSV, tandis que les tables du schéma **CORE** sont enrichies et structurées à travers les transformations réalisées ultérieurement à l'aide de dbt. Cette approche garantit une séparation claire entre les données sources et les données analytiques finales.

3.4 dbt : transformations analytiques

3.4.1 Rôle

La couche de transformation du pipeline ShopStream est assurée par **dbt (data build tool)**. dbt est un outil dédié à la transformation des données analytiques directement au sein du data warehouse Snowflake. Il permet d'implémenter la logique métier de manière structurée, reproductible et versionnée, en s'appuyant exclusivement sur le langage SQL.

dbt s'inscrit dans une approche **ELT**, où les données sont d'abord chargées telles quelles dans Snowflake, puis transformées a posteriori. Cette stratégie permet d'exploiter pleinement la puissance de calcul du data warehouse tout en simplifiant l'architecture globale du pipeline.

3.4.2 Modèles

Les transformations dbt sont organisées sous forme de **modèles SQL**, chacun correspondant à une table analytique. Ces modèles permettent de structurer les données selon des principes de modélisation analytiques clairs, en distinguant notamment :

- les **tables de dimensions** (par exemple `dim_users`, `dim_products`), qui décrivent les entités métier de manière descriptive et relativement stable ;
- les **tables de faits** (par exemple `fact_orders`), qui contiennent les mesures quantitatives utilisées pour l'analyse, telles que les montants de vente ou le nombre de commandes.

Cette modélisation améliore la lisibilité des données, facilite leur compréhension par les analystes et optimise les performances des requêtes exécutées par les outils de Business Intelligence.

```

Which database would you like to use?
[1] snowflake
|
(Don't see the one you want? https://docs.getdbt.com/docs/available-adapters)
|
Enter a number: 1
account (https://<this_value>.snowflakecomputing.com): udkcxdb.ab40596
user (dev username): Marwa
[1] password
[2] keypair
[3] sso
Desired authentication type option (enter a number): 1
password (dev password):
role (dev role): ACCOUNTADMIN
warehouse (warehouse name): LOADING_WH
database (default database that dbt will build objects in): SHOPSTREAM_DWH
schema (default schema that dbt will build objects in): ANALYTICS
threads (1 or more) [1]: 4

```

```

Invite de commandes
(dbt_env) C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt>dbt debug
20:02:06 Running with dbt=1.11.0-rc3
20:02:06 dbt version: 1.11.0-rc3
20:02:06 python version: 3.10.11
20:02:06 python path: C:\Users\LENOVO\dbt_env\Scripts\python.exe
20:02:06 os info: Windows-10-10.0.19044-SP0
20:02:06 Using profiles dir at C:\Users\LENOVO\dbt\profiles
20:02:06 Using profiles.yml file at C:\Users\LENOVO\dbt\profiles.yml
20:02:06 Using dbt_project.yml file at C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt\dbt_project.yml
20:02:06 adapter type: snowflake
20:02:06 adapter version: 1.10.4
20:02:06 Configuration:
20:02:06   profiles.yml file [OK found and valid]
20:02:06   dbt_project.yml file [OK found and valid]
20:02:06 Required dependencies:
20:02:06   - git [OK found]
20:02:06 Connection:
20:02:06   account: udkcxdb-ab40596
20:02:06   user: Marwa
20:02:06   database: SHOPSTREAM_DWH
20:02:06   warehouse: TRANSFORM_WH
20:02:06   role: ACCOUNTADMIN
20:02:07   schema: DBT_DEV
20:02:07   authenticator: None
20:02:07   oauth_client_id: None
20:02:07   query_tag: None
20:02:07   client_session_keep_alive: False
20:02:07   host: None
20:02:07   port: None
20:02:07   proxy_host: None
20:02:07   proxy_port: None
20:02:07   protocol: None
20:02:07   connect_retries: 1
20:02:07   connect_timeout: None
20:02:07   retry_on_database_errors: False
20:02:07   retry_all: False
20:02:07   insecure_mode: False
20:02:07   reuse_connections: True
20:02:07   s3_stage_vpce_dns_name: None
20:02:07   platform_detection_timeout_seconds: 0.0
20:02:07 Registered adapter: snowflake-1.10.4
20:02:09 Connection test: [OK connection ok]
20:02:09 All checks passed!

```

```
(dbt_env) C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt>dbt test
20:24:57 Running with dbt=1.11.0-rc3
20:24:58 Registered adapter: snowflake=1.10.4
20:24:59 Found 13 data tests, 5 models, 4 sources, 504 macros
20:24:59 Concurrency: 4 threads (target='dev')
20:24:59
20:25:01 1 of 13 START test source_not_null_staging_stg_order_items_id ..... [RUN]
20:25:01 2 of 13 START test source_not_null_staging_stg_orders_id ..... [RUN]
20:25:01 3 of 13 START test source_not_null_staging_stg_orders_user_id ..... [RUN]
20:25:01 4 of 13 START test source_not_null_staging_stg_products_id ..... [RUN]
20:25:01 1 of 13 PASS source_not_null_staging_stg_order_items_id ..... [PASS in 0.60s]
20:25:01 2 of 13 PASS source_not_null_staging_stg_orders_id ..... [PASS in 0.60s]
20:25:01 5 of 13 START test source_not_null_staging_stg_users_email ..... [RUN]
20:25:01 4 of 13 PASS source_not_null_staging_stg_products_id ..... [PASS in 0.61s]
20:25:01 6 of 13 START test source_not_null_staging_stg_users_id ..... [RUN]
20:25:01 7 of 13 PASS source_relationships_staging_stg_order_items_order_id_id_source_staging_stg_orders_ [RUN]
20:25:01 3 of 13 PASS source_not_null_staging_stg_orders_user_id ..... [PASS in 0.63s]
20:25:01 8 of 13 START test source_relationships_staging_stg_orders_user_id_id_source_staging_stg_users_ [RUN]
20:25:01 5 of 13 PASS source_not_null_staging_stg_users_email ..... [PASS in 0.16s]
20:25:01 9 of 13 START test source_unique_staging_stg_order_items_id ..... [RUN]
20:25:01 6 of 13 PASS source_not_null_staging_stg_users_id ..... [PASS in 0.18s]
20:25:01 10 of 13 START test source_unique_staging_stg_orders_id ..... [RUN]
20:25:01 7 of 13 PASS source_relationships_staging_stg_order_items_order_id_id_source_staging_stg_orders_ [PASS in 0.17s]
20:25:01 11 of 13 START test source_unique_staging_stg_products_id ..... [RUN]
20:25:01 8 of 13 PASS source_relationships_staging_stg_orders_user_id_id_source_staging_stg_users_ [PASS in 0.18s]
20:25:01 12 of 13 START test source_unique_staging_stg_users_email ..... [RUN]
20:25:02 9 of 13 PASS source_unique_staging_stg_order_items_id ..... [PASS in 0.15s]
20:25:02 13 of 13 START test source_unique_staging_stg_users_id ..... [RUN]
20:25:02 10 of 13 PASS source_unique_staging_stg_orders_id ..... [PASS in 0.16s]
20:25:02 11 of 13 PASS source_unique_staging_stg_products_id ..... [PASS in 0.15s]
20:25:02 12 of 13 PASS source_unique_staging_stg_users_email ..... [PASS in 0.12s]
20:25:02 13 of 13 PASS source_unique_staging_stg_users_id ..... [PASS in 0.14s]
20:25:03
20:25:03 Finished running 13 data tests in 0 hours 0 minutes and 3.85 seconds (3.85s).
20:25:03
20:25:03 Completed successfully
20:25:03
20:25:03 Done. PASS=13 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=13
```

3.4.3 Tests

dbt intègre des **tests automatiques de qualité des données**, permettant de garantir la fiabilité des transformations réalisées. Parmi les tests les plus couramment utilisés figurent :

- l’unicité des clés primaires ;
- la non-nullité des champs critiques ;
- la cohérence des relations entre tables (tests de clés étrangères).

Ces tests sont exécutés à chaque exécution du pipeline et permettent de détecter rapidement les anomalies, réduisant ainsi les risques d’erreurs dans les indicateurs analytiques et les tableaux de bord.

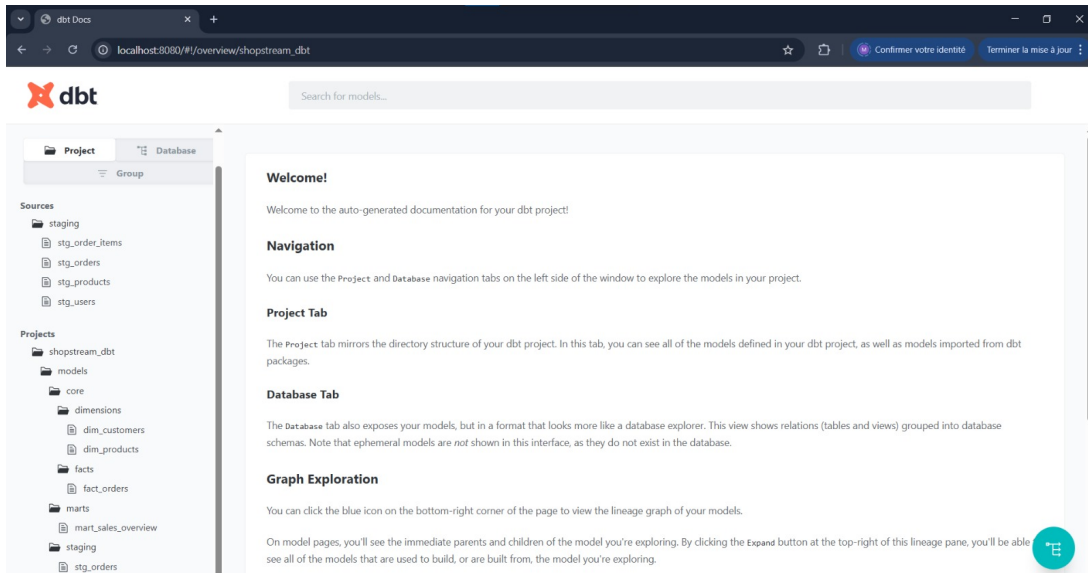
```
(dbt_env) C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt>dbt run
20:20:18 Running with dbt=1.11.0-rc3
20:20:18 Registered adapter: snowflake=1.10.4
20:20:19 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.shopstream_dbt.marts
20:20:19 Found 13 data tests, 4 models, 4 sources, 504 macros
20:20:19 Concurrency: 4 threads (target='dev')
20:20:19
20:20:22 1 of 4 START sql table model DBT_DEV_CORE.dim_customers ..... [RUN]
20:20:22 2 of 4 START sql table model DBT_DEV_CORE.dim_products ..... [RUN]
20:20:22 3 of 4 START sql table model DBT_DEV_CORE.fact_orders ..... [RUN]
20:20:22 4 of 4 START sql view model DBT_DEV_STAGING.stg_orders ..... [RUN]
20:20:23 2 of 4 OK created sql view model DBT_DEV_STAGING.stg_orders ..... [SUCCESS 1 in 0.73s]
20:20:24 2 of 4 OK created sql table model DBT_DEV_CORE.dim_products ..... [SUCCESS 1 in 1.24s]
20:20:24 1 of 4 OK created sql table model DBT_DEV_CORE.dim_customers ..... [SUCCESS 1 in 1.27s]
20:20:24 3 of 4 OK created sql table model DBT_DEV_CORE.fact_orders ..... [SUCCESS 1 in 1.30s]
20:20:25
20:20:25 Finished running 3 table models, 1 view model in 0 hours 0 minutes and 5.50 seconds (5.50s).
20:20:25
20:20:25 Completed successfully
20:20:25
20:20:25 Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=4
```

3.4.4 Documentation

dbt permet également de générer automatiquement une **documentation interactive** décrivant l’ensemble du modèle analytique. Cette documentation inclut la description des modèles, des colonnes, des relations et des dépendances entre tables.

Elle facilite la maintenance du projet, améliore la collaboration entre les équipes techniques et analytiques, et simplifie l’intégration de nouveaux intervenants au sein du projet.

```
(dbt_env) C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt>dbt docs generate
20:25:53 Running with dbt=1.11.0-rc3
20:25:53 Registered adapter: snowflake=1.10.4
20:25:54 Found 13 data tests, 5 models, 4 sources, 504 macros
20:25:54 Concurrency: 4 threads (target='dev')
20:25:54 Building catalog
20:25:57 Catalog written to C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt\target\catalog.json
20:26:04
```



```
Invite de commandes
(dbt_env) C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt>dbt debug
20:02:06 Running with dbt=1.11.0-rc3
20:02:06 dbt version: 1.11.0-rc3
20:02:06 python version: 3.10.11
20:02:06 python path: C:\Users\LENOVO\dbt_env\Scripts\python.exe
20:02:06 os info: Windows-10-10.0.19044-SP0
20:02:06 Using profiles dir at C:\Users\LENOVO\dbt
20:02:06 Using profiles.yml file at C:\Users\LENOVO\dbt\profiles.yml
20:02:06 Using dbt_project.yml file at C:\Users\LENOVO\Documents\Marwa\ShopStreamTP\shopstream_dbt\dbt_project.yml
20:02:06 adapter type: snowflake
20:02:06 adapter version: 1.10.4
20:02:06 Configuration:
20:02:06   profiles.yml file [OK found and valid]
20:02:06   dbt_project.yml file [OK found and valid]
20:02:06 Required dependencies:
20:02:06   - git [OK found]
20:02:06 Connection:
20:02:06   account: udkcxdab-ab40596
20:02:06   user: Marwa
20:02:06   database: SHOPSTREAM_DWH
20:02:06   warehouse: TRANSFORM_WH
20:02:06   role: ACCOUNTADMIN
20:02:06   schema: DBT_DEV
20:02:06   authenticator: None
20:02:06   oauth_client_id: None
20:02:06   query_tag: None
20:02:06   client_session_keep_alive: False
20:02:06   host: None
20:02:06   port: None
20:02:06   proxy_host: None
20:02:06   proxy_port: None
20:02:06   protocol: None
20:02:06   connect_retries: 1
20:02:06   connect_timeout: None
20:02:06   retry_on_database_errors: False
20:02:06   retry_all: False
20:02:06   insecure_mode: False
20:02:06   reuse_connections: True
20:02:06   s3_stage_vpce_dns_name: None
20:02:06   platform_detection_timeout_seconds: 0.0
20:02:06 Registered adapter: snowflake=1.10.4
20:02:09 Connection test: [OK connection ok]
20:02:09 All checks passed!
```



```

(dbt_env) C:\Users\LENOVO\Documents\Warwa\ShopStreamTP\shopstream_dbt>dbt run
20:20:18 Running with dbt=1.11.0-rc3
20:20:18 Registered adapter: snowflake=1.10.4
20:20:19 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.shopstream_dbt.marts
20:20:19 Found 13 data tests, 4 models, 4 sources, 504 macros
20:20:19 Concurrency: 4 threads (target='dev')
20:20:19
20:20:22 1 of 4 START sql table model DBT_DEV_CORE.dim_customers ..... [RUN]
20:20:22 2 of 4 START sql table model DBT_DEV_CORE.dim_products ..... [RUN]
20:20:22 3 of 4 START sql table model DBT_DEV_CORE.fact_orders ..... [RUN]
20:20:22 4 of 4 START sql view model DBT_DEV_STAGING.stg_orders ..... [RUN]
20:20:23 4 of 4 OK created sql view model DBT_DEV_STAGING.stg_orders ..... [SUCCESS 1 in 0.73s]
20:20:24 2 of 4 OK created sql table model DBT_DEV_CORE.dim_products ..... [SUCCESS 1 in 1.24s]
20:20:24 1 of 4 OK created sql table model DBT_DEV_CORE.dim_customers ..... [SUCCESS 1 in 1.27s]
20:20:24 3 of 4 OK created sql table model DBT_DEV_CORE.fact_orders ..... [SUCCESS 1 in 1.30s]
20:20:25
20:20:25 Finished running 3 table models, 1 view model in 0 hours 0 minutes and 5.50 seconds (5.50s).
20:20:25
20:20:25 Completed successfully
20:20:25
20:20:25 Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=4

```

3.5 Airflow : orchestration du pipeline

3.5.1 Rôle

Apache Airflow est utilisé comme **outil d'orchestration** du pipeline de données ShopStream. Son rôle principal est de coordonner, planifier et automatiser l'exécution des différentes étapes du pipeline, depuis l'extraction des données transactionnelles jusqu'à la production des données analytiques finales.

Airflow n'effectue pas de transformations directes sur les données. Il agit comme un **chef d'orchestre**, chargé de déclencher les traitements au bon moment, dans le bon ordre, tout en assurant le suivi de leur exécution. Cette orchestration garantit la cohérence globale du pipeline et permet son industrialisation.

3.5.2 Concept de DAG

Dans Airflow, un pipeline est défini sous la forme d'un **DAG (Directed Acyclic Graph)**, c'est-à-dire un graphe orienté sans cycle. Chaque nœud du DAG représente une tâche, tandis que les arêtes définissent les dépendances entre ces tâches.

Cette représentation permet de modéliser précisément l'ordre d'exécution des différentes étapes du pipeline et d'empêcher toute exécution incohérente. Une tâche ne peut être lancée que si toutes les tâches dont elle dépend ont été exécutées avec succès.

```
nadaezzouhour — zsh — 80x39
Last login: Sat Dec 13 13:00:11 on console
(base) nadaezzouhour@NADAC ~ % python3.11 -m venv airflow_env

(base) nadaezzouhour@NADAC ~ % source airflow_env/bin/activate

(airflow_env) (base) nadaezzouhour@NADAC ~ % airflow version
2.8.0
(airflow_env) (base) nadaezzouhour@NADAC ~ % python3.11 -m venv airflow_env

(airflow_env) (base) nadaezzouhour@NADAC ~ % source airflow_env/bin/activate

(airflow_env) (base) nadaezzouhour@NADAC ~ % rm -rf ~/ShopStreamTP/airflow

(airflow_env) (base) nadaezzouhour@NADAC ~ % mkdir -p ~/ShopStreamTP/airflow
export AIRFLOW_HOME=~/ShopStreamTP/airflow

(airflow_env) (base) nadaezzouhour@NADAC ~ % echo $AIRFLOW_HOME
/Users/nadaezzouhour/ShopStreamTP/airflow
(airflow_env) (base) nadaezzouhour@NADAC ~ % airflow db init

/Users/nadaezzouhour/airflow_env/lib/python3.11/site-packages/airflow/cli/commands/db_command.py:47 DeprecationWarning: `db init` is deprecated. Use `db migrate` instead to migrate the db and/or airflow connections create-default-connections to create the default connections
DB: sqlite:///Users/nadaezzouhour/ShopStreamTP/airflow/airflow.db
[2025-12-13T13:09:41.151+0100] {migration.py:213} INFO - Context impl SQLiteImpl.
[2025-12-13T13:09:41.153+0100] {migration.py:216} INFO - Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running stamp_revision -> 10b52ebd31f7
WARNI [airflow.models.crypto] empty cryptography key - values will not be stored encrypted.
Initialization done
(airflow_env) (base) nadaezzouhour@NADAC ~ %
```

3.5.3 Structure du DAG ShopStream

Le DAG mis en place pour le pipeline ShopStream est composé de plusieurs tâches successives :

- **Extraction des données depuis PostgreSQL** : récupération des données transactionnelles (utilisateurs, commandes, produits) à partir de la base OLTP.

Warning: Fields that are currently populated can be modified but cannot be deleted. To delete data from a field, delete the Connection object and create a new one.

Add Connection

Connection Id * postgres_shopstream

Connection Type * Postgres
Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.

Description

Host localhost

Database shopstream

Login postgres

Password

Port 5432

Extra

[Save](#) [Test](#) [←](#)

- **Chargement des données vers Amazon S3** : conversion des données extraites en fichiers CSV et stockage dans la zone RAW du Data Lake.

The screenshot shows the 'Add Connection' form in the Airflow web interface. The browser address bar indicates the URL is `192.168.2.39:8080/connection/add`. The form fields are as follows:

- Connection Id ***: `aws_default`
- Connection Type ***: `Amazon Web Services`. Below the dropdown, a message reads: "Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package."
- Description**: (Empty text area)
- AWS Access Key ID**: `AKIAIPP4WARDP3LZA364`
- AWS Secret Access Key**: (Masked with dots)
- Extra**: `{"region_name": "eu-west-3"}`

At the bottom of the form, there are buttons for **Save**, **Test**, and a back arrow. On the right side of the form, there are three circular icons: a gear, a plus sign, and a play button.

- **Chargement des données dans Snowflake** : importation des fichiers stockés dans S3 vers les tables du schéma STAGING à l'aide de commandes `COPY INTO`.

The screenshot shows the 'Add Connection' page in the Airflow web interface. The browser address bar indicates the URL is `192.168.2.39:8080/connection/add`. The Airflow navigation bar at the top includes links for DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The form fields are as follows:

- Connection Id**: `snowflake_default`
- Connection Type**: `Snowflake` (with a dropdown arrow and a note: "Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.")
- Description**: (empty text area)
- Schema**: `STAGING`
- Login**: `Marwa`
- Password**: (masked with dots)
- Extra**: `{"private_key_file": "private key", "session_parameters": "session parameters"}`
- Account**: `ab40596.eu-west-3.aws`
- Warehouse**: `LOADING_WH`
- Database**: `SHOPSTREAM_DWH`
- Region**: `snowflake hosted region`
- Role**: `ACCOUNTADMIN`

— **Exécution des transformations dbt** : application de la logique métier et création des tables analytiques finales dans le schéma CORE.

Chaque tâche dépend explicitement de la précédente, assurant ainsi un enchaînement logique et sécurisé des traitements.

3.5.4 Gestion des dépendances et des échecs

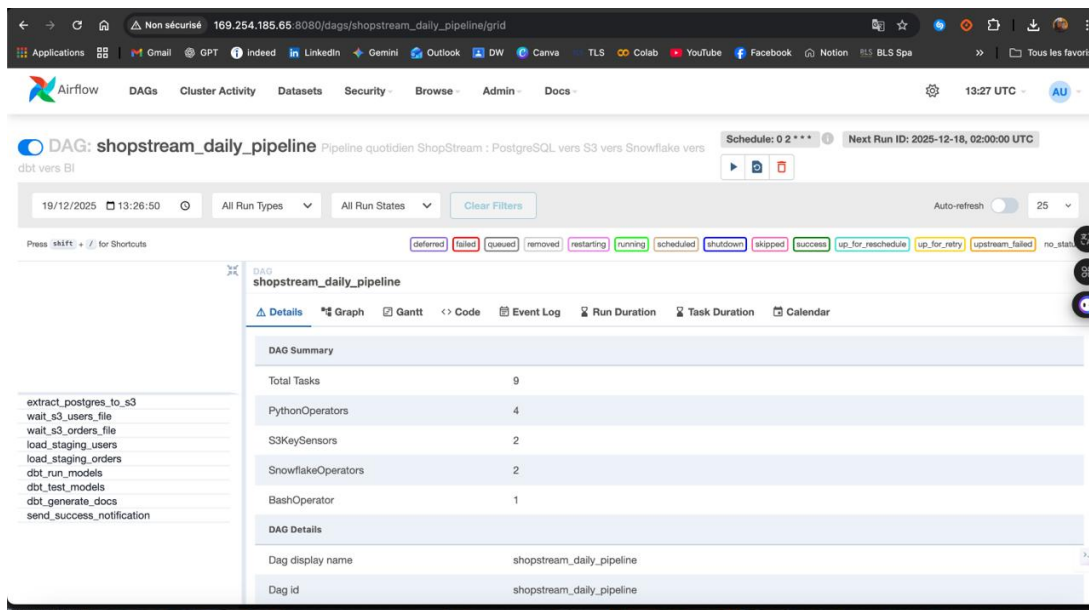
Airflow permet de définir des dépendances strictes entre les tâches du DAG. Ainsi, si une étape échoue, les étapes suivantes ne sont pas exécutées. Cette mécanique empêche la propagation d'erreurs et garantit que seules des données complètes et cohérentes sont utilisées pour les analyses.

En cas d'échec, Airflow enregistre les logs d'exécution et offre la possibilité de relancer uniquement les tâches concernées, sans avoir à rejouer l'intégralité du pipeline. Cette capacité améliore la robustesse et la maintenabilité du système.

3.5.5 Planification et automatisation

Airflow permet également de planifier l'exécution du pipeline selon une fréquence définie (quotidienne, horaire ou hebdomadaire). Cette automatisation garantit la mise à jour régulière des données analytiques sans intervention manuelle.

Grâce à cette planification, les tableaux de bord Power BI reposent toujours sur des données fraîches et fiables, ce qui est essentiel pour la prise de décision.



3.5.6 Apport d’Airflow dans l’architecture globale

L’intégration d’Airflow dans le pipeline ShopStream apporte plusieurs avantages majeurs :

- automatisation complète du pipeline de données ;
- gestion centralisée des dépendances ;
- traçabilité et historisation des exécutions ;
- amélioration de la fiabilité et de la reproductibilité des traitements.

Airflow joue ainsi un rôle fondamental dans la transformation d’un ensemble de scripts indépendants en un **pipeline de données industriel, robuste et scalable**.

3.6 Power BI : visualisation et analyse décisionnelle

3.6.1 Rôle

Power BI constitue la **couche finale de visualisation** du pipeline de données ShopStream. Son rôle est de restituer les données analytiques, préalablement traitées et structurées dans Snowflake, sous forme de tableaux de bord interactifs destinés aux utilisateurs métiers et aux décideurs.

Contrairement aux autres composants du pipeline, Power BI ne stocke pas les données et n’effectue pas de transformations lourdes. Il se connecte directement aux tables analytiques finales produites par dbt dans Snowflake, garantissant ainsi que les visualisations reposent sur des données fiables, cohérentes et conformes aux règles métier définies en amont.

3.6.2 Connexion aux données

Dans l’architecture ShopStream, Power BI est connecté au data warehouse Snowflake, qui constitue la source unique de vérité pour l’analyse décisionnelle. Cette connexion permet d’exécuter des requêtes analytiques performantes tout en évitant toute surcharge de la base transactionnelle PostgreSQL.

Cette approche assure une séparation claire entre les traitements analytiques et la couche de visualisation, et permet aux utilisateurs métiers d’accéder aux indicateurs sans nécessiter de compétences techniques avancées.

3.6.3 Dashboards

Les tableaux de bord développés avec Power BI offrent une vision synthétique et interactive de l’activité de la plateforme ShopStream. Ils permettent notamment de visualiser :

- le **chiffre d'affaires par pays**, afin d'identifier les zones géographiques les plus rentables ;
- l'**évolution temporelle des ventes**, permettant d'analyser les tendances, la saisonnalité et les périodes de forte activité ;
- les **produits les plus performants**, facilitant les décisions liées à la gestion du catalogue ;
- la **répartition et le comportement des clients**, offrant une meilleure compréhension du profil des utilisateurs.

Les dashboards Power BI sont interactifs et permettent l'application de filtres dynamiques (par période, pays ou produit), offrant ainsi une exploration flexible des données.

3.6.4 Mesures DAX

Le calcul des indicateurs clés de performance repose sur l'utilisation du langage **DAX**. Les mesures DAX permettent de définir des calculs dynamiques qui s'adaptent automatiquement aux filtres appliqués dans les tableaux de bord.

Dans le cadre du projet ShopStream, des mesures DAX sont utilisées pour calculer notamment :

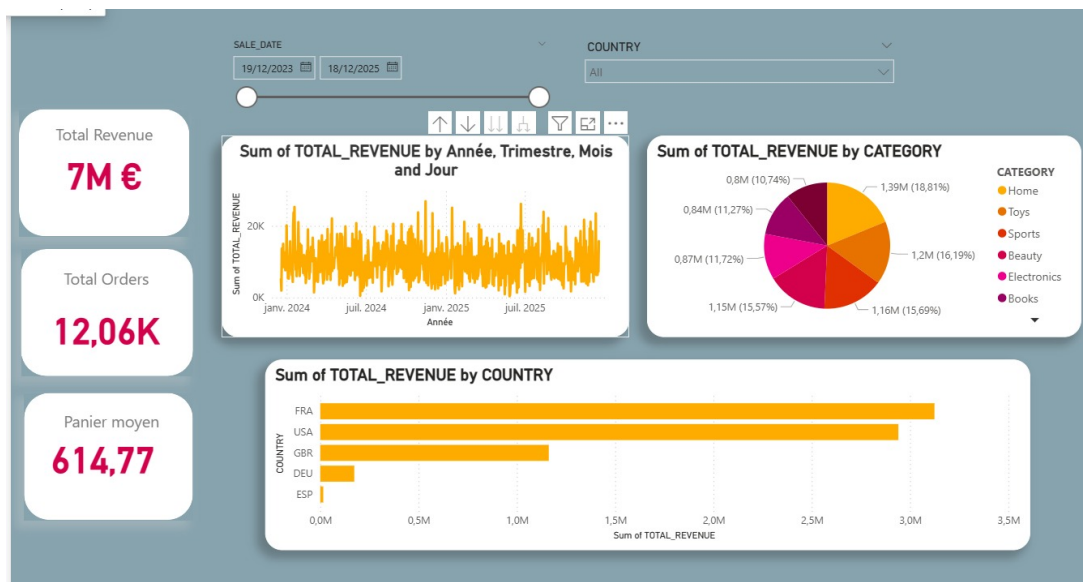
- le **chiffre d'affaires total**, obtenu par l'agrégation des montants des commandes ;
- le **nombre de commandes**, indicateur du volume d'activité commerciale ;
- le **panier moyen**, calculé comme le rapport entre le chiffre d'affaires total et le nombre de commandes.

Ces mesures permettent une analyse fine et dynamique des performances de la plateforme, sans nécessiter de recalculs au niveau du pipeline technique.

3.6.5 Apport de Power BI dans l'architecture globale

L'intégration de Power BI dans le pipeline ShopStream permet de transformer les données analytiques en **valeur métier concrète**. Les tableaux de bord facilitent le suivi des indicateurs clés, améliorent la compréhension des performances de l'activité et soutiennent la prise de décision stratégique.

Power BI constitue ainsi l'aboutissement du pipeline de données, en assurant la transition entre les traitements techniques et l'exploitation métier des données.



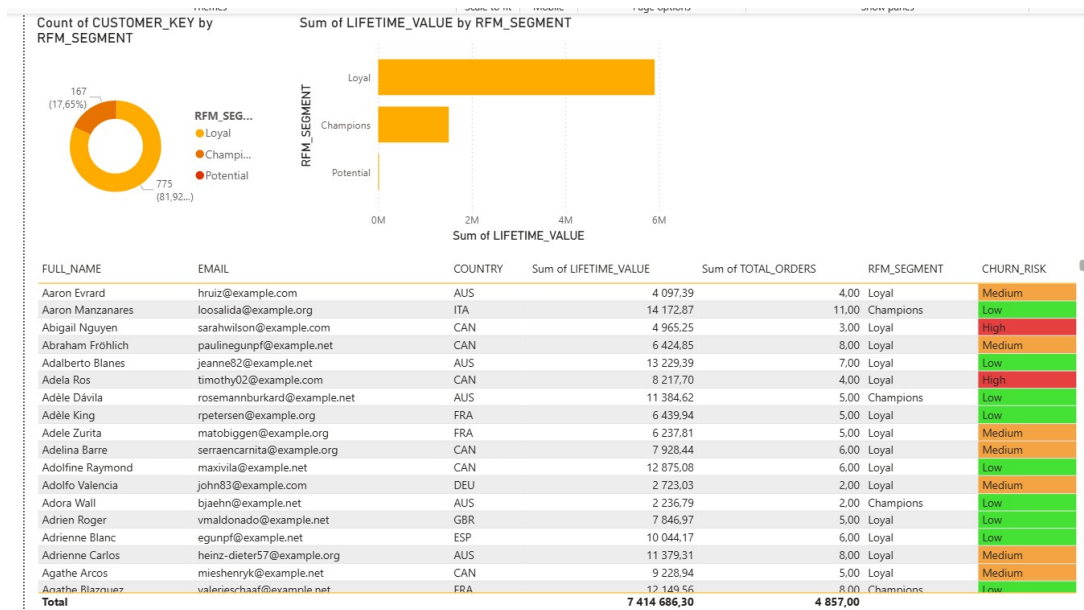


FIGURE 3.1 – Analyse des clients

PRODUCT_NAME	CATEGORY	Sum of TOTAL_ORDERS	Sum of TOTAL_REVENUE	ABC_CLASS	PERFORMANCE_TIER
Triple-buffered static utilization	Fashion	69.00	776.79	C	Long Tail
Configurable multi-state analyzer	Sports	62.00	1 103.30	C	Long Tail
Down-sized fault-tolerant hardware	Beauty	66.00	1 210.32	C	Long Tail
Le confort d'innover plus rapidement	Books	64.00	1 523.97	C	Long Tail
Customer-focused 5thgeneration installation	Electronics	56.00	1 719.86	C	Long Tail
Operative homogeneous moderator	Sports	74.00	2 083.14	C	Long Tail
Reduced encompassing customer loyalty	Electronics	65.00	2 806.02	C	Long Tail
Balanced interactive secured line	Books	77.00	3 148.50	C	Long Tail
Customer-focused asymmetric Internet solution	Home	70.00	3 912.75	C	Long Tail
Triple-buffered context-sensitive focus group	Beauty	79.00	3 932.10	C	Long Tail
Monitored explicit instruction set	Home	73.00	4 658.19	C	Long Tail
Robust asynchronous matrices	Electronics	58.00	5 125.12	C	Long Tail
Ameliorated dedicated utilization	Sports	74.00	5 841.63	C	Long Tail
Reactive mission-critical toolset	Electronics	63.00	5 956.47	C	Long Tail
La liberté de concrétiser vos projets plus facilement	Fashion	65.00	6 298.05	C	Long Tail
Virtual full-range analyzer	Books	60.00	6 914.40	C	Long Tail
Ergonomic 5thgeneration project	Fashion	80.00	7 476.81	C	Long Tail
Le pouvoir d'avancer à la pointe	Beauty	57.00	7 754.06	C	Long Tail
Le droit d'évoluer autrement	Fashion	75.00	8 737.83	C	Long Tail
Profit-focused multimedia projection	Home	58.00	8 795.20	C	Long Tail
De-engineered content-based focus group	Electronics	73.00	9 288.52	C	Long Tail
Diverse secondary definition	Toys	69.00	9 339.84	C	Long Tail
Total fresh-thinking task-force	Toys	71.00	9 380.00	C	Long Tail
Monitored actuating algorithm	Sports	63.00	9 437.22	C	Long Tail
Cross-group bifurcated adapter	Toys	71.00	9 854.41	C	Long Tail
Intuitive full-range open system	Beauty	84.00	10 226.72	C	Long Tail
Le plaisir d'évoluer avant-tout	Toys	59.00	10 254.44	C	Long Tail
L'avantage d'innover à sa source	Toys	63.00	11 320.38	C	Long Tail
Cloned national success	Electronics	70.00	11 468.88	C	Long Tail
Implemented bottom-line moratorium	Electronics	73.00	11 494.30	C	Long Tail
Expanded zero-defect parallelism	Fashion	66.00	12 438.80	C	Long Tail
Total		14 559,00	7 414 686,30		

FIGURE 3.2 – Performance des Produits

Conclusion

Ce projet a permis de concevoir et de mettre en œuvre un **pipeline de données moderne** pour la plateforme fictive ShopStream, en s'appuyant sur des outils et des pratiques largement utilisés dans les environnements professionnels. À travers ce TP, nous avons couvert l'ensemble du cycle de vie de la donnée, depuis sa génération dans une base transactionnelle jusqu'à son exploitation sous forme de tableaux de bord décisionnels.

3.7 Bilan du projet

Le pipeline mis en place repose sur une architecture claire et modulaire, inspirée du *Modern Data Stack*. PostgreSQL a été utilisé comme source transactionnelle pour stocker les données métier en temps réel. Ces données ont ensuite été exportées et centralisées dans un Data Lake Amazon S3, garantissant leur conservation brute et leur traçabilité. Snowflake a joué le rôle de Data Warehouse analytique, permettant l'exécution de requêtes complexes sur de grands volumes de données.

Les transformations analytiques ont été réalisées à l'aide de dbt, assurant une application cohérente et versionnée des règles métier, tandis qu'Apache Airflow a permis d'orchestrer et d'automatiser l'ensemble du pipeline. Enfin, Power BI a servi de couche de visualisation, offrant des tableaux de bord interactifs facilitant l'analyse des performances de la plateforme et la prise de décision.

Ce projet a permis de mettre en pratique des notions essentielles telles que la séparation OLTP/OLAP, l'approche ELT, l'orchestration de workflows, la qualité des données et la visualisation décisionnelle. Il constitue ainsi une reproduction réaliste d'une architecture de données moderne utilisée en entreprise.

3.8 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours de la réalisation de ce projet. La première concerne la **compréhension et l'intégration de plusieurs technologies** au sein d'une même architecture, chacune ayant ses propres concepts et contraintes. La configuration des connexions entre les différents outils (PostgreSQL, S3, Snowflake, dbt et Airflow) a également représenté un défi technique important.

Par ailleurs, la gestion de la qualité des données et la cohérence des transformations analytiques ont nécessité une attention particulière, notamment lors de la mise en place des modèles dbt et des tests associés. L'orchestration du pipeline avec Airflow a également demandé un effort de conception afin de définir correctement les dépendances entre les tâches et de gérer les cas d'échec.

Enfin, la conception de tableaux de bord Power BI pertinents a impliqué une réflexion approfondie sur le choix des indicateurs clés de performance et sur la manière de restituer efficacement l'information aux utilisateurs métiers.

3.9 Améliorations possibles

Bien que le pipeline mis en place soit fonctionnel et conforme aux objectifs du TP, plusieurs améliorations peuvent être envisagées. Tout d'abord, la mise en place de **mécanismes avancés de qualité des données** (alertes automatiques, suivi des anomalies, indicateurs de qualité) permettrait de renforcer la fiabilité globale du système.

Ensuite, l'architecture pourrait être enrichie par l'ajout de **gestion des accès et de la sécurité avancée**, notamment à travers le contrôle d'accès granulaire (Row-Level Security) et la prise en compte plus approfondie des exigences

réglementaires telles que le RGPD. L'automatisation du déploiement du pipeline à l'aide de pratiques DevOps ou CI/CD constituerait également une amélioration significative.

Enfin, le pipeline pourrait être étendu pour intégrer des traitements en quasi temps réel, l'analyse prédictive ou l'apprentissage automatique, afin d'exploiter davantage la valeur des données collectées. Ces évolutions permettraient de rapprocher encore davantage ce projet d'une architecture de données complète et industrielle.

En conclusion, ce TP constitue une base solide pour comprendre les enjeux et les pratiques des architectures de données modernes. Il met en évidence l'importance d'un pipeline bien structuré pour transformer des données brutes en informations à forte valeur ajoutée au service de la décision.